

정비기록 공통원인 후보 탐지 시스템 — 기술서

이 문서는 비전공자도 읽을 수 있도록 작성했습니다. 전문 용어가 나올 때마다 그 자리에서 풀어 설명하고, 맨 뒤에 용어 사전을 두었습니다. 작성일: 2026-09-22 · 대상 저장소: jjhokim/failure-dashboard

0. 한눈에 보기

한 문장 요약

정비병이 손으로 적은 정비기록 수백~수천 건을 읽어서, "이거 같은 원인 아닌가?" 싶은 고장들을 자동으로 묶어주고, 어느 묶음부터 손봐야 할지 순서를 매겨주는 도구를 만들고, 그 도구가 진짜 작동하는지를 스스로를 속이지 않는 방법으로 검증했습니다.

무엇을 만들었나

결과물	내용
분석 도구	고장기록을 입력·조회·분석하는 웹 대시보드 (7 개 화면)

결과물	내용
탐지 엔진	문장 의미 + 부품 위치 + 발생 시각을 함께 보고 고장을 묶는 알고리즘
우선순위 엔진	묶인 고장군에 점수를 매겨 순위표를 만드는 기능
검증 체계	가짜 데이터를 만들어 도구 성능을 측정하되, 자기가 자기를 채점하는 함정 을 차단한 설계
실험 결과	사전에 합격기준을 공표하고 8 번 반복 실험한 결과 보고서 (불합격 항목 포함)

가장 정직하게 말해야 할 결과

사전에 정한 6 개 합격기준 중 **5 개는 통과, 1 개는 불합격**했습니다. 불합격 항목을 숨기거나 기준을 나중에 바꾸지 않고 그대로 보고했습니다. 그리고 그 불합격의 원인을 추적한 결과, **도구의 문제가 아니라 시험 문제(가짜 데이터)의 설계 문제**였다는 것을 밝혀냈습니다.

1. 배경 — 왜 이걸 만들어야 했나

1.1 현장에서 벌어지는 일

유도무기 같은 무기체계는 한 번 배치되면 수십 년을 운용합니다. 그동안 부품이 고장나고, 정비병이 고치고, 그 내용을 기록으로 남깁니다. 기록은 대략 이렇게 생겼습니다.

발생일시: 2025-04-11 12:36

체계명: 체계-B

부품: LRU-010

고장유형: 기계적

고장증상: "구동축 마모로 인한 위치 오차 발생" ← 사람이 쓴 자유서술

조치내용: "기어 어셈블리 교환 후 백래시 조정"

여기서 **핵심 정보**는 "고장유형: 기계적"이 아니라 "구동축 마모로 인한 위치 오차 발생"이라는 문장 안에 있습니다. 그런데 이 문장은 사람마다 다르게 씁니다.

- A 정비병: "구동모터에서 과열 발생"
- B 정비병: "구동모터 온도 급상승 확인"
- C 정비병: "고온 환경에서 구동모터 발열 이상"

세 사람이 같은 고장을 적었는데 글자는 하나도 안 겹칩니다. 그래서 "과열"로 검색하면 B와 C는 안 걸립니다. 이게 첫 번째 문제입니다.

1.2 진짜 위험한 상황: 공통원인 고장

개별 고장 하나 하나는 대수롭지 않을 수 있습니다. 문제는 **여러 고장이 사실은 하나의 뿌리에서 나온 경우**입니다. 예를 들면,

- 특정 시기에 납품된 부품 로트(batch)에 불량이 섞여 있었다
- 어떤 창고의 보관 환경(고온·다습)이 부품을 열화시켰다
- 설계상 특정 조건에서만 드러나는 결함이 있다

이런 걸 **공통원인 고장(Common-Cause Failure, CCF)**이라고 합니다. 개별 고장으로 보면 "그냥 가끔 고장나는구나" 싶지만, 묶어서 보면 "아, 이건 2024년 3월 로트 문제구나"가 드러납니다.

공통원인을 못 찾으면 어떻게 되나? - 같은 고장이 계속 재발합니다. 부품만 갈아 끼우고 근본 원인은 그대로이기 때문입니다. - 정비 비용과 불가동 시간이 누적됩니다. - 최악의 경우, 결정적 순간에 여러 장비가 동시에 먹통이 됩니다.

1.3 그런데 왜 지금까지 못 했나

선행 연구¹가 지적하듯, 현장 데이터 수집 자체가 만만치 않습니다.

- 수기로 작성되어 전산화가 안 된 기록
- 담당자마다 다른 작성 방식
- 인수인계 누락
- 사업이 끝나면 수집도 끊김

즉 "어떤 데이터를 어떻게 모을 것인가" 가 선행 과제였습니다. 그런데 데이터를 모으고 나면 다음 질문이 옵니다. "모았으니, 이제 이걸로 뭘 하지?"

이 프로젝트는 바로 그 다음 단계입니다.

2. 아이디어 — 핵심 착상

2.1 사람은 어떻게 공통원인을 찾는가

숙련된 IPS 담당자가 기록 더미를 보고 공통원인을 의심할 때, 머릿속에서는 대략 세 가지를 동시에 봅니다.

1. "증상이 비슷한데?" — 문장의 의미
2. "부품 위치가 가깝네?" — 구조상 이웃인가
3. "시기가 몰려 있네?" — 비슷한 시점에 터졌나

셋 중 하나만으로는 부족합니다. - 증상만 보면: "과열"은 전혀 다른 원인으로도 생깁니다. - 위치만 보면: 같은 부품이라고 다 같은 원인은 아닙니다. - 시기만 보면: 우연히 같은 주에 터진 무관한 고장도 많습니다.

그런데 셋이 동시에 겹치면? 그때 비로소 "같은 원인 아닐까?" 하는 의심이 강해집니다.

2.2 그래서 이렇게 만들었다

이 직관을 그대로 코드로 옮겼습니다.

고장기록 A와 고장기록 B가 얼마나 "가까운가"를 세 가지 잣대로 각각 채고, 세 점수를 합쳐서 하나의 "종합 거리"를 만든다.
그 다음 서로 가까운 것들끼리 자동으로 묶는다.

여기서 "거리"라는 말이 핵심입니다. 두 기록이 비슷하면 거리가 0에 가깝고, 전혀 다르면 1에 가깝습니다. 세 개의 거리를 각각 구한 뒤 가중평균을 냅니다.

종합거리 = $0.5 \times (\text{문장 의미 거리})$
 + $0.25 \times (\text{부품 위치 거리})$
 + $0.25 \times (\text{발생 시각 거리})$

가중치(0.5 / 0.25 / 0.25)는 "의미를 절반, 나머지 둘을 각각 4분의 1씩 보겠다"는 뜻입니다. 어떤 채널의 가중치를 0으로 두면 그 채널을 아예 안 보는 것이 되는데, 이 성질을 나중에 "그 채널이 정말 필요한가?"를 시험하는 데 씁니다.

2.3 묶는 것으로 끝이 아니다 — 순서를 매겨야 한다

고장 600건을 돌리면 군집이 10개쯤 나옵니다. 그럼 10개를 다 조사해야 하나? 현실적으로 인력과 예산이 한정되어 있으니, 어느 것부터 볼지 순서가 필요합니다.

세 가지 축으로 점수를 냅니다.

측	의미	왜 중요한가
규모	이 묶음에 고장이 몇 건 들어있나	많이 터질수록 영향이 크다
추세	최근에 늘고 있나, 줄고 있나	늘고 있으면 급하다
치명도	임무에 지장을 주는 고장 비율이 높은가	못 쓰게 만드는 고장이 더 위험하다

세 점수를 표준화해서 합산하고, 높은 순으로 순위표를 만듭니다. 순위표에는 **대표 부품 위치, 대표 문장 3 개**를 같이 보여줘서 담당자가 바로 "아 이런 건이구나" 파악할 수 있게 했습니다.

3. 가장 중요한 문제 — "자기가 자기를 채점하는 함정"

이 절이 이 프로젝트의 **핵심 기여**입니다. 조금 어렵지만 비유로 풀어보겠습니다.

3.1 문제 상황

도구를 만들었으면 "**얼마나 잘 맞이나?**" 를 측정해야 합니다. 그런데 실제 야전 데이터에는 **정답이 없습니다**. "이 10 건이 진짜 같은 원인이었다"고 알려주는 사람이 없으니까요.

그래서 흔히 쓰는 방법이 **가짜 데이터(합성데이터) 만들기**입니다.

1. 내가 직접 "이 10 건은 원인 A" "저 8 건은 원인 B" 라고 정해서 가짜 고장기록을 만든다.
2. 도구에게 그 기록을 주고 묶어보라고 한다.
3. 도구가 묶은 결과와 내가 정한 정답을 비교해서 점수를 낸다.

합리적으로 보입니다. 그런데 여기에 함정이 있습니다.

3.2 함정: 시험문제에 답을 적어놓기

가짜 데이터를 만들 때 이런 실수를 하기 쉽습니다.

원인 A 그룹의 고장 문장을 만들 때, 문장에 "A"라는 표시를 슬쩍 남긴다. 예를 들어 A 그룹은 전부 "과열"이라는 단어를 쓰고, B 그룹은 전부 "진동"을 쓰게 한다.

그러면 도구는 "과열"이라는 단어만 보고도 **100 점**을 받습니다. 하지만 이건 도구가 똑똑해서가 아니라, **시험지에 답이 적혀 있었기 때문**입니다.

이걸 **순환검증(circular validation)** 또는 "**double dipping**" 이라고 합니다. 신경과학 분야에서 유명한 연구 [2](#)가 이 문제를 지적했는데, 그 연구는 충격적인 실험을 했습니다.

아무 효과도 없는 데이터(사람이 아무것도 안 하고 누워 있을 때의 뇌 영상)에 당시 널리 쓰이던 분석 절차를 그대로 적용했더니, **있지도 않은 효과가 검출되었다**. 그리고 2008 년 주요 학술지 논문 134 편을 조사했더니 **42%가 이 함정에 빠져 있었다**.

3.3 우리는 이렇게 차단했다

"정답 라벨은 절대로 관측 데이터에 닿지 않는다" 는 원칙을 세우고, 세 가지 규칙으로 강제했습니다.

규칙 1 — 정답은 문장 생성기를 거치지 않는다

가짜 문장을 만드는 과정을 2 단계로 쪼갬니다.

[1 단계] 정답 라벨 → 물리 상태로 변환 ← 사람이 짤 규칙 코드만 사용
"원인 A" → {부품: 구동모터, 증상: 과열, 환경: 고온, 시각: 3월 12일}

[2 단계] 물리 상태 → 자연스러운 문장 생성 ← 여기서 라벨은 아예 안 보임
{구동모터, 과열, 고온} → "고온 환경에서 구동모터 과열이 발생함"

2 단계 함수는 "원인 A"라는 정보를 인자로 받지도 못합니다. 코드 수준에서 막아줬고, 실수로 넣으면 에러가 나도록 했습니다. 즉 "정답은 문장 만드는 단계를 거치지 않는다".

규칙 2 — 만드는 쪽과 찾는 쪽이 도구를 공유하지 않는다

가짜 데이터를 만드는 프로그램과 공통원인을 찾는 프로그램이 같은 단어 사전이나 같은 분석 도구를 쓰면, 찾는 쪽이 만드는 쪽의 규칙을 그대로 되읽게 됩니다.

그래서 두 프로그램은 서로를 참조하지 못하도록 분리했고, 단어 사전도 따로 만들었습니다. (자동 검사로 강제합니다 — 만약 누가 나중에 코드를 고쳐서 서로 참조하게 하면 테스트가 실패합니다.)

규칙 3 — 자체진단 로그는 탐지에 안 쓴다

장비에는 BIT(Built-In Test)라는 자체진단 기능이 있어서 이상이 생기면 코드를 남깁니다. 이걸 탐지에 쓰면 도움이 될 것 같지만, **가짜 BIT 로그는 결국 우리가 정답을 알고 만든 것이라** 또 순환이 생깁니다.

그래서 BIT 는 "이 고장 근처에 이런 진단 신호가 있었다"고 보여주기만 하고, 묶는 계산에는 절대 넣지 않았습니다.

그리고 — 두 개의 대조군

위 세 규칙을 지켜도 불안하니, 일부러 아무 신호도 없는 데이터로 시험했습니다.

대조군	방법	기대 결과	실제 결과
무주입	공통원인을 하나도 안 심은 데이터	아무것도 못 찾아야 정상	점수 -0.002 (사실상 0) ☒
라벨 순열	정답표를 무작위로 섞어버림	점수가 0 으로 붕괴해야 정상	점수 -0.003 ☒

두 대조군 모두 통과했습니다. 이건 3.2 에서 소개한 연구가 한 일(효과 없는 데이터로 허위 효과를 점검)과 정확히 같은 논리입니다.

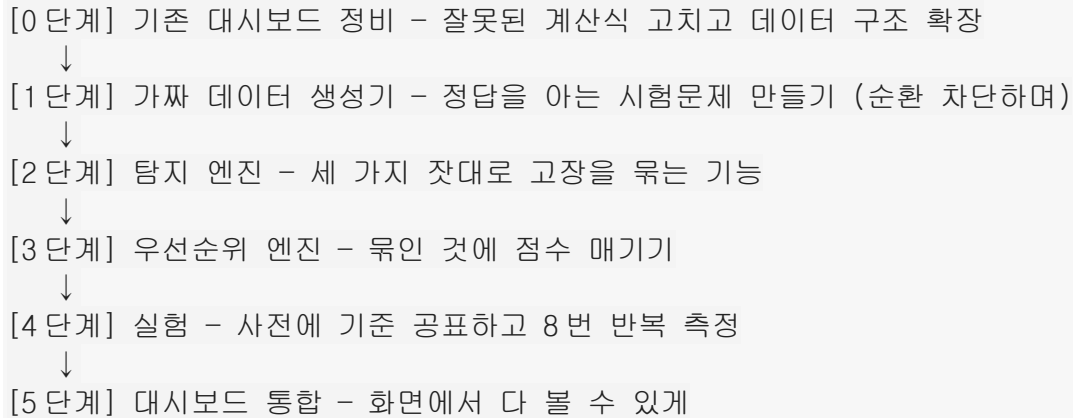
추가로 — 누출 자동 검사 3 종

데이터를 만들 때마다 자동으로 세 가지를 검사합니다.

1. 문장에 "군집", "cluster", "라벨" 같은 금지 단어가 들어갔나 → **0%여야 함**
2. 문장만 보고 정답을 맞히는 간단한 기계를 학습시키면 얼마나 맞히나 → 무주입 대비 비교
3. 그룹끼리 단어가 얼마나 겹치나

4. 프로젝트 흐름 — 무엇을 어떤 순서로 만들었나

4.1 전체 그림



4.2 0 단계 — 기존 코드의 계산 오류 3 건 수정

프로젝트 시작 시점의 대시보드에는 **지표 계산에 오류가 3 개** 있었습니다.

오류 1: 이름이 틀렸다 (Ao 라고 썼는데 실은 Ai)

화면에 "**가동률 Ao**" 라고 표시하고 있었는데, 실제 계산식은 Ao 가 아니라 **Ai** 였습니다.

- **Ai (고유가용도)**: 순수하게 "고장나는 빈도"와 "고치는 시간"만 따진 값. 설계 단계에서 쓰는 이상적인 수치입니다.
- **Ao (운용가용도)**: 여기에 **부품 기다리는 시간, 행정 절차 시간** 등 현실의 지연까지 포함한 값. 실제 운용에서 의미 있는 건 이쪽입니다.

Ao 는 항상 Ai 보다 낮습니다. 그런데 Ai 를 Ao 라고 부르면 "**우리 장비 가동률 87%**" 라고 보고하게 되는데 실제 운용 가동률은 그보다 훨씬 낮을 수 있습니다. **위험한 착각**입니다.

→ 이름을 Ai 로 정정하고, 진짜 Ao 는 필요한 데이터(정비지연시간)가 없으면 **아예 계산하지 않고 "미산출"로 표시**하도록 했습니다. 억지로 숫자를 만들지 않는 것이 더 정직하기 때문입니다.

오류 2: 평균 수리시간을 짧게 잡고 있었다

MTTR(평균 수리시간)을 계산할 때 "수리 완료된 건"만 평균을 냈습니다. 언뜻 당연해 보이지만 함정이 있습니다.

쉬운 고장은 빨리 끝나서 "완료"로 집계됩니다. 어려운 고장은 아직도 수리 중이라 집계에서 빠집니다. 결과: 쉬운 것만 평균내니 수리시간이 실제보다 짧게 나옵니다.

이걸 통계학에서 **우측절단(right censoring)** 문제라고 합니다. 예를 들어,

- 완료된 수리: 2 시간, 4 시간 → 단순 평균 **3 시간**
- 그런데 아직 진행 중인 수리가 하나 있고 벌써 10 시간째다
- 이걸 반영하면 실제 평균은 **5.3 시간**

의료 통계에서 "환자 생존기간"을 다룰 때 쓰는 **카플란-마이어(Kaplan-Meier)** 방법을 그대로 적용했습니다. "아직 안 끝난 것도 최소 이만큼은 걸렸다"는 정보를 버리지 않고 반영하는 방법입니다.

실제 데이터에 적용해 보니: 단순 평균 20.8 시간 → 보정 후 **973.6 시간**. 47 배 차이입니다. 수리 중인 11 건이 232~8,077 시간(약 10 일~11 개월) 동안 열려 있었기 때문입니다. 기존 방식은 이 사실을 완전히 가리고 있었습니다.

오류 3: MTBF 분모가 잘못됐다

MTBF(평균 고장간격)를 "**달력상 경과 시간 ÷ 고장 건수**" 로 계산하고 있었습니다. 문제는 장비가 창고에 가만히 있어도 달력은 흘러간다는 것입니다.

→ "**실제 운용한 시간의 합 ÷ 고장 건수**" 로 바꿨습니다. 운용시간 데이터가 없으면 기존 방식으로 계산하되, 화면에 "**달력근사**" 라고 표시해서 이 값이 근사치임을 숨기지 않도록 했습니다.

4.3 1 단계 — 가짜 데이터 생성기

3 장에서 설명한 순환 차단 원칙을 지키면서 가짜 정비기록을 만듭니다.

조절할 수 있는 손잡이(파라미터)

손잡이	뜻	예
기록 수	몇 건 만들까	600 건
군집 수	공통원인을 몇 개 심을까	10 개
신호 강도 δ	공통원인을 얼마나 뚜렷하게 심을까	0.0 ~ 1.0
잡음 비율	아무 관계 없는 단발 고장 비율	20%
표현 편차	같은 증상을 얼마나 다르게 쓸까	0.5

신호 강도 δ 가 핵심입니다. - $\delta = 0$: 공통원인을 하나도 안 심음 $\rightarrow$ 도구가 아무것도 못 찾아야 정상 (대조군) - $\delta = 1.0$: 아주 뚜렷하게 심음 $\rightarrow$ 도구가 잘 찾아야 정상

δ 를 0 에서 1 까지 올리면서 "신호가 강해질수록 점수도 올라가는가" 를 봅니다. 점수가 δ 와 무관하게 높다면 뭔가 잘못된 것(순환 의심)입니다.

4.4 2 단계 — 탐지 엔진

세 가지 거리를 각각 계산합니다.

① 의미 거리 — 문장이 얼마나 비슷한가

한국어 문장을 숫자 벡터로 바꾸는 **KoSBERT** 라는 인공지능 모델을 씁니다. 비슷한 뜻의 문장은 비슷한 숫자로 변환되는 성질이 있습니다.

실제로 확인해 봤습니다.

"구동모터에서 과열이 발생함" ↔ "구동모터 발열 이상 확인" → 거리 0.18 (가깝다)
"구동모터에서 과열이 발생함" ↔ "통신모듈 신호 끊김" → 거리 0.77 (멀다)

글자는 거의 안 겹치는데도 **뜻이 비슷하면 가깝다고 판단합니다**. 1.1 절에서 말한 "사람마다 다르게 쓰는" 문제를 이걸로 해결합니다.

② 구조 거리 — 부품 위치가 얼마나 가까운가

부품에는 **LCN(군수관리번호)**이라는 계층 주소가 붙습니다. 주소 체계와 비슷합니다.

A.01.03.02 → A 체계 / 1 번 계통 / 3 번 조립체 / 2 번 부품

두 부품이 **얼마나 가까운 친척인가**를 계산합니다. - A.01.03.02 와 A.01.03.05 → 형제 (가깝다) - A.01.03.02 와 B.05.09.01 → 완전 남남 (멀다)

③ 시간 거리 — 발생 시점이 얼마나 가까운가

시간거리 = $1 - \exp(-|\text{시간차}| / 30 \text{ 일})$

같은 날 발생하면 0 에 가깝고, 몇 달 차이냐면 1 에 가까워집니다.

묶기 — HDBSCAN

세 거리를 합친 뒤 **HDBSCAN**이라는 알고리즘으로 묶습니다. 이 알고리즘을 고른 이유는 두 가지입니다.

1. **몇 개로 묶을지 미리 안 정해도 됩니다.** (실제로 공통원인이 몇 개인지 모르니까)
2. **"어디에도 안 속함"을 인정합니다.** 단발성 고장을 억지로 어딘가에 끼워넣지 않고 **잡음(-1)**으로 따로 빼줍니다. 정비기록에는 진짜 우연한 고장이 많으므로 중요합니다.

4.5 3~5 단계

- **3 단계:** 묶인 군집에 규모·추세·치명도 점수를 매겨 순위표 생성
- **4 단계:** 실험 (5 장에서 자세히)

- 5 단계: 대시보드에 화면 3 개 추가 — 공통원인 후보 / 실험 결과 / BIT 연결

5. 실험 — 무엇을 알아냈나

5.1 먼저: 실험 전에 기준을 공표했다

이 프로젝트에서 가장 신경 쓴 부분입니다.

왜 미리 정해야 하나? 실험을 해보고 나서 기준을 정하면, 무의식적으로라도 "내 결과가 통과하도록" 기준을 맞추게 됩니다. 이걸 막으려면 **결과를 보기 전에** 합격선을 문서로 못박아야 합니다.

그래서 `preregistration.md` 라는 문서에 6 개 기준을 적고 **커밋(기록 고정)**한 뒤 실험을 돌렸습니다. 그리고 **실행 후 그 문서를 수정하지 않았습니다.**

또한 모든 수치는 **8 번 반복(시드 8 개)한 평균 $\pm$ 표준편차**로만 보고합니다. 한 번 돌려서 좋은 결과가 나온 걸 인용하는 것은 금지했습니다.

5.2 결과 — 5 개 통과, 1 개 불합격

#	기준	결과	판정
C1	공통원인을 안 심으면 아무것도 못 찾아야	점수 -0.002, 잡음비율 0.83	☑

#	기준	결과	판정
C2	정답표를 섞으면 점수가 0 으로 붕괴해야	-0.003	☑
C3	신호가 강할수록 점수가 올라가야	단조 증가	☑
C4	세 채널을 합친 게 하나만 쓴 것보다 나아야	합친 것 0.736 < 시간만 0.853	✕
C5	순위가 정답 순위와 상관 있어야	0.526, 신뢰구간 [0.267, 0.760]	☑
C6	점수 축들의 상관관계 보고	규모~치명도 = +0.841	⚠ 보고

C4가 불합격입니다. 세 채널을 합쳐 쓰는 게 이 프로젝트의 핵심 아이디어인데, 시간 하나만 쓴 것보다 못했습니다.

5.3 불합격을 어떻게 처리했나 — 기준을 바꾸지 않았다

여기서 유혹이 생깁니다. "기준을 좀 낮추면 통과인데..." 또는 "이 실험은 없던 걸로 하고 다시 돌릴까?"

둘 다 하지 않았습니다. 불합격은 불합격으로 보고하고, 대신 원인을 조사했습니다. 그리고 원인 조사는 새로운 사전등록 문서를 다시 쓰고 진행했습니다.

가설 1: 가중치 배분이 나빴나? → 틀렸습니다

혹시 0.5/0.25/0.25 라는 가중치가 나빠서일까 싶어, 가능한 모든 가중치 조합을 다 시험해 봤습니다. (단, 가중치를 고르는 데이터와 성능을 재는 데이터를 완전히 분리해서 또 다른 순환을 막았습니다.)

결과: 최적 조합이 "시간만 1.0, 나머지 0" 이었습니다. 즉 자유롭게 고르게 했더니 기계가 스스로 나머지 두 채널을 꺼버렸습니다. 가중치 문제가 아니었습니다.

가설 2: 시험문제가 잘못됐나? → 맞았습니다

가짜 데이터를 만들 때 "같은 원인이면 비슷한 시기에 터진다"고 설정했는데, 그 "비슷한 시기"의 폭이 $\delta=1.0$ 에서 하루로 줄어들도록 코드가 짜여 있었습니다.

같은 원인의 고장이 전부 하루 안에 몰려 있으면? 날짜만 봐도 다 맞힐 수 있습니다. 문장이고 부품이고 볼 필요가 없습니다.

즉 시험문제가 시간 채널에게만 지나치게 유리하게 출제되어 있었습니다.

그래서 이 폭을 하루 → 3.65 일(관측기간의 1%)로 현실적으로 완화하고 다시 돌렸습니다.

	세 채널 합침	시간만
기존 (하루로 몰림)	0.736	0.853 ← 시간이 이김
수정 (3.65 일로 분산)	0.938	0.786 ← 합친 게 이김

역전되었습니다. 게다가 이 조건에서 가중치를 다시 자유 탐색했더니 이번엔 세 채널을 모두 살렸습니다 (0.5 / 0.25 / 0.25 — 원래 기본값과 같음).

그래서 결론은

"세 채널을 합치는 게 좋은가?"의 답은 "경우에 따라 다르다"입니다. 공통 원인 고장이 하루 안에 다 물려서 터지면 날짜만 봐도 되니 합칠 필요가 없습니다. 그러나 며칠~몇 주에 걸쳐 퍼져서 나타나면 세 채널을 합치는 쪽이 확실히 낫습니다.

그리고 실제 현장이 어느 쪽인지는 이 연구가 답하지 못합니다. (7 장 한계 참조)

5.4 민감도 분석 — 경계가 어디인가

"며칠부터 합치는 게 유리해지나?"를 확인하려고, 시간 폭을 1 일부터 30 일까지 바꿔가며 전부 측정했습니다.

시간 폭	세 채널 합침	시간만	누가 이기나
1 일	0.685	0.848	시간
2 일	0.877	0.831	무승부
3.65 일	0.938	0.786	합침
7 일	0.940	0.776	합침
14 일	0.889	0.631	합침
30 일	0.881	0.411	합침 (격차 최대)

경계는 2~3.65 일 사이입니다. 그리고 시간 폭이 넓어질수록 격차가 더 벌어집니다.

이 실험의 신뢰도를 보여주는 증거 하나: 표에는 안 넣었지만, "의미만 쓴 경우"와 "부품위치만 쓴 경우"의 점수는 1 일부터 30 일까지 전 구간에서 정확히 똑같았습니다(0.232, 0.197). 당연합니다 — 시간 폭을 바꿨으니 시간 채널만 영향을 받아야 하니까요. 이게 그대로 확인되었다는 건 실험이 의도한 것만 건드렸다는 뜻입니다.

5.5 언제쯤 알아챌 수 있나 (탐지 지연)

실무에서 중요한 질문입니다. "고장이 몇 건쯤 쌓여야 공통원인이 보이나요?"

측정 결과 (600 건 중):

신호 강도	찾아낸 비율	필요한 누적 고장 건수
약함 ($\delta=0.25$)	7%	478 건
중간 ($\delta=0.50$)	28%	309 건
강함 ($\delta=1.00$)	56%	269 건

신호가 아무리 강해도 절반 정도만, 그것도 600 건 중 269 건(45%)이 쌓인 뒤에야 알아챱니다.

이 결과의 솔직한 의미: 이 도구는 "조기경보 장치"가 아닙니다. 고장이 터지자마자 알려주는 게 아니라, 어느 정도 쌓인 뒤에 "돌이켜보니 이것들이 한 묶음이었네"를 찾아주는 사후 분석 도구입니다. 이 점을 오해하면 안 됩니다.

5.6 제가 두 번 틀렸던 이야기 (C6)

기술서에 실패담을 적는 건 이례적이지만, **교훈이 명확해서** 남깁니다.

"규모" 점수와 "치명도" 점수가 서로 중복되는 것 아닌가 하는 의심이 있었습니다. 판단이 세 번 바뀌었습니다.

1. **이론적 추론**: "계산식을 보니 중복이 맞다" → **근거 없이 단정한 오류**
2. **소규모 예비실험 1 회**: 상관 +0.20 → "중복 아니네, 기각" → **1 회 실험으로 판단한 오류**
3. **본 실험 8 회**: 상관 +0.841 → "역시 중복이 맞나?"
4. **최종 검증**: 중복을 실제로 제거해 봤더니 → **순위 정확도가 0.920 → 0.750 으로 악화**

결론: 그 상관은 제거해야 할 중복이 아니라 실제로 유용한 신호였습니다. 정답 치명도 자체가 "묶음 안의 값을 다 더한 것"이라 규모와 상관되는 게 당연했습니다.

교훈: 한 번 돌린 실험 결과로 판단하면 안 된다는 원칙은 성능 수치뿐 아니라 진단용 통계에도 똑같이 적용되어야 합니다. 저는 예비실험 1 회 결과(+0.20)로 사전등록 문서에 잘못된 근거를 적었습니다. 다행히 **결정 자체는 옳았지만 근거가 틀렸었고**, 이를 보고서에 그대로 밝혔습니다.

5.7 재현성 확인

같은 설정으로 실험을 다시 돌려, **120 개 조건 × 시드 조합 전체**에서 결과가 일치하는지 확인했습니다.

결과: 모든 항목의 차이가 정확히 0.0. 난수를 전부 통제된 설계가 실제로 작동함을 확인했습니다. 즉 누가 언제 돌려도 같은 숫자가 나옵니다.

6. 활용 기술 — 무엇을 썼고 왜 골랐나

6.1 기반 기술

기술	역할	왜 이것 골랐나
Python	전체 개발 언어	데이터 분석 생태계가 가장 풍부
Streamlit	웹 대시보드	파이썬만으로 화면을 만들 수 있어 별도 웹 개발이 불필요
SQLite	데이터 저장	서버 설치 없이 파일 하나로 동작. 로컬 단일 PC 환경에 적합
pandas	표 형태 데이터 처리	사실상 표준
Plotly	대화형 그래프	마우스로 확대·탐색 가능

6.2 분석 기술

KoSBERT (한국어 문장 임베딩)

하는 일: 한국어 문장을 숫자 목록(벡터)으로 바꿉니다. 뜻이 비슷하면 숫자도 비슷합니다.

쉬운 비유: 도서관에서 책을 주제별로 쪼개는 것과 비슷합니다. 제목의 글자가 달라도 내용이 비슷하면 가까운 서가에 꽂힙니다. KoSBERT 는 문장에 대해 그 "서가 위치"를 숫자로 계산해 줍니다.

쓴 모델: `snunlp/KR-SBERT-V40K-k1ueNLI-augSTS` (서울대 NLP 연구실 공개 모델)

HDBSCAN (밀도 기반 군집화)

하는 일: 가까운 것들끼리 자동으로 묶습니다.

쉬운 비유: 밤하늘의 별을 보고 별자리를 찾는 것과 비슷합니다. 별이 뽁뽁하게 모인 곳은 하나의 무리로 보고, 혼자 외따로 떨어진 별은 **"어느 무리에도 안 속함"** 으로 남겨둡니다.

왜 이걸 골랐나 - 몇 개로 묶을지 미리 안 정해도 됨 (실제로 공통원인이 몇 개인지 모름) - **잡음을 인정함** — 단발 고장을 억지로 끼워넣지 않음 - 미리 계산한 거리표를 그대로 넣을 수 있음 (우리는 3 채널을 합친 거리를 쓰므로 필수)

구현 참고: 별도의 `hdbscan` 패키지 대신 `scikit-learn` 에 내장된 것을 썼습니다. 알고리즘은 같으면서 설치 시 컴파일이 필요 없어 윈도우 환경에서 문제가 없기 때문입니다.

Kaplan-Meier (생존분석)

하는 일: "아직 안 끝난 것"이 섞여 있어도 평균을 제대로 추정합니다.

쉬운 비유: 병원에서 "이 병의 평균 생존기간"을 계산할 때, 아직 살아 계신 분들을 빼고 계산하면 실제보다 짧게 나옵니다. 그렇다고 넣을 수도 없죠(아직 모르니까). Kaplan-Meier 는 **"최소한 지금까지는 살아 계셨다"**는 정보를 버리지 않고 반영합니다.

우리는 이걸 **수리시간**에 적용했습니다. "아직 수리 중"인 건을 버리지 않습니다.

그 외

기술	역할
TF-IDF	단어 빈도 기반 고전적 방법. 비교 기준선 과 누출 검사 에 사용
ARI	두 묶음 결과가 얼마나 일치하는지 재는 척도. 0 이면 우연 수준, 1 이면 완전 일치
Spearman 상관	두 순위표가 얼마나 비슷한지 재는 척도
부트스트랩	적은 표본으로 "이 차이가 우연일 가능성"을 추정하는 방법
playwright	대시보드 화면을 자동으로 캡처 (논문 그림용)

6.3 개발 품질 관리

항목	내용
자동 테스트 121 건	코드를 고칠 때마다 전부 돌려서 기존 기능이 깨지지 않았는지 확인
작은 단위 커밋	기능 하나당 하나씩 기록을 남겨 나중에 되짚을 수 있게

항목	내용
난수 통제	무작위가 들어가는 모든 곳에 "씨앗값"을 넣어 항상 같은 결과가 나오게
사전등록 문서	실험 기준을 먼저 적고 고정

7. 활용 방안 — 실제로 어떻게 쓰나

7.1 지금 바로 쓸 수 있는 것

① 정비기록 관리 대시보드

7 개 화면으로 구성되어 있습니다.

화면	용도
요약 대시보드	전체 현황 한눈에 — 가용도, MTBF, MTTR, 미완료 건수
고장 입력	새 고장기록 등록

화면	용도
고장 현황	조건별 조회, 수정, 삭제
분석	파레토 분석 (어떤 부품·유형이 문제인지 상위부터)
공통원인 후보	자동 군집화 + 우선순위 순위표
실험 결과	검증 실험 결과 시각화
BIT 연결	자체진단 로그와 정비기록 연결 조회

② 외부 데이터 가져오기

엑셀이나 CSV 파일을 올리면 자동으로 컬럼을 맞춰 넣습니다. 한국어 컬럼명(제대구분, 발생일시)과 영어 컬럼명(echelon, occurred_at) 둘 다 인식하고, **맞출 수 없는 컬럼은 경고로 알려줍니다.**

③ 지표 계산 엔진

Ai, MTBF(운용시간 기준), MTTR(절단 보정) 계산을 **검증된 함수**로 제공합니다. 계산식과 근거가 코드 주석에 적혀 있어 감사·검토가 가능합니다.

7.2 실무 시나리오

시나리오 A — 분기별 공통원인 검토 회의

1. 지난 분기 정비기록을 CSV 로 내려받아 업로드

2. [공통원인 후보] 화면에서 탐지 실행
3. 순위표 상위 3개 군집 확인
→ 각 군집의 대표 부품 위치와 대표 문장 3개를 보고
"아, 이건 B.01 계통 제어보드 문제 같다" 파악
4. 해당 군집을 클릭해 상세 목록 확인
5. 의심되는 건을 FMECA · 고장분석 절차로 넘김

중요: 이 도구는 "여기부터 보세요" 라고 후보를 좁혀줄 뿐, 원인을 판정하지 않습니다. 판정은 사람이 정식 절차로 해야 합니다.

시나리오 B — 가중치 조정으로 다른 관점 보기

화면의 슬라이더로 가중치를 바꿀 수 있습니다. - 의미 가중치를 높이면 → **증상이 비슷한 것** 위주로 묶임 - 구조 가중치를 높이면 → **같은 계통 부품** 위주로 묶임 - 시간 가중치를 높이면 → **비슷한 시기 발생** 위주로 묶임

같은 데이터를 여러 관점에서 보면서 담당자의 직관과 대조할 수 있습니다.

7.3 연구·문서화 용도

- **논문 작성:** 그림 6종, 표 5종을 명령어 한 줄로 재생성
- **재현성:** 누가 언제 돌려도 같은 결과 (검증 완료)
- **방법론 참고:** 순환검증 차단 설계는 **다른 분야의 합성데이터 평가에도 그대로 적용** 가능합니다. 이것이 이 프로젝트의 가장 이전 가능한 자산일 수 있습니다.

8. 기대 효과

8.1 직접 효과

효과	설명
검토 대상 축소	수백 건을 다 읽는 대신 상위 몇 개 군집부터 검토
표현 차이 극복	"과열"과 "온도 급상승"을 같은 것으로 인식
다차원 근거	"증상도 비슷하고 부품도 이웃이고 시기도 물렸다"는 복합 근거 제시
지표 신뢰도 회복	Ai/Ao 혼동, 수리시간 과소평가 등 기존 오류 3 건 수정

8.2 간접 효과

데이터 품질 개선 유인

이 도구는 LCN(부품 주소)이 있어야 제 성능이 납니다. 즉 **"LCN 을 제대로 기록하면 이런 분석이 가능하다"** 는 것을 보여줌으로써, 현장에서 데이터를 제대로 적을 동기를 만듭니다. 선행 연구 [1](#)가 지적한 데이터 품질 문제에 대한 **구체적 유인**입니다.

정직한 지표 문화

- 계산할 수 없는 지표는 **억지로 만들지 않고 "미산출"로 표시**
- 모든 지표 카드에 **데이터 출처 배지** 표시 (샘플/합성/외부, 섞였으면 비율까지)
- MTTR 에 **완료/미완료 건수 병기** — 편향 가능성을 숨기지 않음

검증 방법론 자산

3 장의 순환검증 차단 설계는 이 프로젝트만의 것이 아닙니다. **"AI 로 뭔가 찾아냈다"**고 주장하는 모든 연구가 같은 함정에 빠질 수 있습니다. 그 함정을 코드 수준에서 막는 구체적 방법을 문서와 테스트로 남겼습니다.

8.3 기대하면 안 되는 것 (중요)

정직하게 선을 긋겠습니다.

기대하면 안 되는 것	이유
✕ 고장 원인을 판정해 준다	후보만 좁혀줍니다. 원인 판정은 FMECA·FTA·고장분석의 몫
✕ 고장을 미리 예측한다	예측 기능은 없습니다. 이미 일어난 고장을 묶을 뿐
✕ 조기에 경보를 준다	5.5 절 — 600 건 중 269 건 쌓인 뒤에야, 그것도 56%만
✕ 실제 야전에서 이만큼 나온다	모든 수치는 가짜 데이터 기준입니다

9. 한계 — 무엇을 못 하는가

9.1 가장 큰 한계: 실제 데이터로 검증하지 않았다

모든 성능 수치는 합성데이터(가짜 데이터) 기준입니다.

- 가짜 데이터는 제가 정한 규칙대로 만들어졌습니다.
- 실제 정비병이 쓰는 문장은 훨씬 다양하고 지저분할 것입니다.
- 오타, 약어, 은어, 빈칸이 실제로는 많을 것입니다.

따라서 "실제 현장에서 몇 % 맞는다"는 말은 이 연구로는 할 수 없습니다.

9.2 결론이 가정에 의존한다

5.3~5.4 절에서 본 대로, "세 채널을 합치는 게 유리한가"의 답이 시간 폭 가정에 따라 뒤집힙니다.

- 1~2 일 가정: 시간 하나만 쓰는 게 낫다
- 3.65 일 이상 가정: 합치는 게 낫다

문제는 어느 쪽이 현실인지 모른다는 것입니다. 문헌을 찾아봤지만, 공통원인 고장의 시간 창은 **표준화된 값이 아니라 "임무와 점검주기에 따라 분석자가 선택하는 값"** 이라고 명시되어 있습니다³. 즉 찾을 수 있는 정답이 없습니다.

그래서 하나를 고르는 대신 **1 일~30 일 전 구간의 결과를 다 보고했습니다**. 정직한 처리이지만, "그래서 현실은 어느 쪽인가"에 답하지 못한다는 한계는 그대로입니다.

9.3 문장 생성기가 진짜 LLM 이 아니다

가짜 정비기록의 문장을 만들 때, 실제 대규모 언어모델(LLM) 대신 **템플릿 + 동의어 치환** 방식을 썼습니다. (오프라인에서도 재현 가능하게 하려고)

그 결과 문장이 실제보다 **규칙적**입니다. 실제로 이 영향이 의심되는 징후가 있습니다. - 단순 키워드 규칙 방식이 점수 0.644 를 받았는데, 이는 **예상보다 높습니다**. - 생성된 문장이 규칙 사전으로 잡기 쉬운 구조였을 가능성이 있습니다.

9.4 탐지 지연 지표가 채널을 구별하지 못한다

7 개 채널 조합별로 탐지 지연을 재봤더니, **군집화 품질이 4 배 넘게 차이 나는데도 지연과 발견율은 거의 같았습니다.**

채널	군집화 점수	발견율	지연
시간만	0.848	0.41	279 건
세 채널	0.685	0.54	292 건
부품위치만	0.197	0.49	298 건

품질이 가장 나쁜 방식(0.197)이 가장 좋은 방식과 비슷한 지연을 보입니다.

원인을 들여다보니, 지연을 판정하는 기준("묶음 하나라도 순도 80% 넘으면 발견")이 **전체 품질을 반영하지 못하기 때문**으로 보입니다. 초기 구간이 한 군집에 쏠려 있으면 어떤 방식이든 쉽게 통과합니다.

다만 이 원인 분석은 한 번의 관찰에 근거한 부분 검증이라 단정하지 않았습니다 (5.6 절의 교훈 때문입니다). 지표 정의를 고치려면 **사전등록을 다시 하고** 재실험해야 합니다.

9.5 기타 한계

한계	내용
낮은 신호는 못 찾음	신호 강도 0.25 이하에서는 점수가 0. 탐지 가능 구간은 0.5 이상
보수적 군집화	뭉은 건 대체로 맞지만(정밀도 0.986) 놓치는 게 많음 (재현율 0.795)
단일 PC 전제	다중 사용자, 권한 관리, 네트워크 보안 미구현
Ao 미산출	정비지연시간(MDT/ALDT) 데이터가 없어 진짜 운용가용도는 계산 불가
샘플 데이터에 LCN 없음	기존 샘플 100 건에는 부품 주소가 없어, 공통원인 기능은 합성/외부 데이터 필요

10. 앞으로 할 일

10.1 가장 시급한 것

실제 야전 정비기록 확보. 9.1~9.2 의 한계가 모두 여기서 풀립니다. 실데이터가 있으면 공통원인 고장이 실제로 며칠에 걸쳐 퍼지는지를 측정할 수 있고, 그러면 9.2 의 "어느 쪽이 현실인가" 질문에 답할 수 있습니다.

10.2 중기 과제

과제	내용
탐지 지연 지표 재정의	9.4 의 문제 해결 (사전등록 후 재실험 필요)
실제 LLM 적용	9.3 의 문장 규칙성 문제 완화
Ao 산출	정비지연시간 필드를 실제로 수집하면 가능
다중 사용자	인증·권한·감사 로그

10.3 하지 않기로 한 것

- 고장 예측 AI: 데이터가 충분히 쌓이기 전에는 시기상조
- BIT 로그를 탐지에 융합: 순환성 위험(3.3 규칙 3) — 의도적으로 배제

11. 용어 사전

용어	쉬운 설명
IPS / ILS	종합군수지원. 장비를 도입부터 폐기까지 굴리는 데 필요한 모든 지원 활동
MRO	정비·수리·운영(Maintenance, Repair, Operation)
RAM	신뢰도·가용도·정비도
Ai (고유가용도)	고장 빈도와 수리 시간만으로 계산한 이상적 가동률
Ao (운용가용도)	부품 대기, 행정 지연까지 포함한 실제 가동률. 항상 Ai 보다 낮음
MTBF	평균 고장간격. 얼마나 자주 고장나나
MTTR	평균 수리시간. 고치는 데 얼마나 걸리나
MDT / ALDT	정비중단시간 / 행정·군수지연시간
LRU	현장에서 통째로 갈아끼우는 부품 단위
LCN	군수관리번호. 부품의 계층 주소 (예: A.01.03.02)

용어	쉬운 설명
CCF (공통원인 고장)	여러 고장이 하나의 뿌리 원인에서 나온 것
FMECA / FTA	고장 원인을 정식으로 분석하는 표준 절차
BIT	장비 자체진단 기능
정비 3 제대	1 제대 부대정비 / 2 제대 야전정비 / 3 제대 창정비
임베딩	문장이나 단어를 숫자 목록으로 바꾼 것
군집화(클러스터링)	비슷한 것끼리 자동으로 묶는 것
HDBSCAN	밀도 기반 군집화 알고리즘. 잡음을 인정함
ARI	두 묶음 결과의 일치도. 0=우연 수준, 1=완전 일치
우측절단	"아직 안 끝난" 데이터가 섞여 통계가 왜곡되는 현상
Kaplan-Meier	우측절단을 보정하는 통계 방법

용어	쉬운 설명
순환검증 / double dipping	정답을 문제에 섞어놓고 채점해서 실제보다 좋아 보이는 함정
사전등록	실험 전에 합격기준을 공개 문서로 고정하는 것
시드(seed)	무작위를 재현 가능하게 만드는 씨앗값
파레토 분석	상위 몇 개가 전체의 대부분을 차지하는지 보는 분석

12. 부록 — 프로젝트 구조

```

failure-dashboard/
├── backend/           데이터 저장 · 조회 · 지표 계산
│   ├── models.py     데이터 구조 정의
│   ├── database.py   DB 연결, 자동 마이그레이션
│   ├── crud.py       입력 · 조회 · 집계
│   ├── metrics.py    Ai · MTBF · MTTR 계산 (순수 함수)
│   ├── constants.py  정비 제대 명칭 (한 곳에서 관리)
│   └── adapters/     엑셀 · CSV 가져오기
├── frontend/
│   └── app.py         대시보드 화면 7 개
├── synth/            가짜 데이터 생성기 (순환 차단)
│   ├── physical.py   [규칙] 정답 → 물리상태
│   ├── narrator.py   [생성] 물리상태 → 문장 (정답 안 보임)
│   ├── leakage.py    누출 검사 3 종
│   └── bit.py         모의 BIT 로그
└── detect/           탐지 엔진

```

		channels.py	3 채널 거리 계산
		fusion.py	거리 합치기
		cluster.py	HDBSCAN 군집화
		baselines.py	비교 기준선 2 종
		prioritize/	
		scoring.py	우선순위 점수 · 순위표
		experiments/	실험 · 평가 · 문서
		preregistration*.md	사전등록 (실행 전 고정)
		results_report*.md	결과 보고서
		run*.py	실험 실행기
		export*.py	논문 그림 · 표 생성
		tests/	자동 테스트 121 건
		docs/	
		기술서.md	(이 문서)

주요 명령어

```
# 대시보드 실행
streamlit run frontend/app.py

# 전체 테스트
python -m pytest tests/ -q

# 실험 재현
python -m experiments.run --config experiments/configs/ablation.yaml --seeds 0-7

# 논문 그림 · 표 생성
python -m experiments.export_figures
```

이 문서는 프로젝트의 실제 코드와 실험 결과에 근거해 작성되었습니다. 불합격 결과와 작성자의 판단 오류를 포함해, 확인된 사실만 기술했습니다.

1. 김민준 외, "유도무기 고장 데이터 기반 MRO 데이터 수집 방향성 연구", 한국산학기술학회논문지, 2025. [↩↩](#)
2. Kriegeskorte 외, "Circular analysis in systems neuroscience: the dangers of double dipping", Nature Neuroscience, 2009. [↩](#)

3. NUREG/CR-6268, NUREG/CR-5485 (미국 원자력규제위원회 CCF 분석
지침) [↩](#)